Zhiyuan Pan

pan2013e.github.io

zy_pan@zju.edu.cn

Google Scholar | GitHub

Education

- **Zhejiang University** — Sep. 2023 – Present
  Ph.D. Student in Artificial Intelligence — Hangzhou, China
  - College of Artificial Intelligence. Advisors: Xing Hu, Xin Xia and Xiaohu Yang.
  - Research interests: AI for Software Engineering, trustworthy agents, and code LLMs.
  - National Scholarship, Ministry of Education, P.R.C. (2024).
- **Zhejiang University, Chu Kochen Honors College** — Sep. 2019 – Jun. 2023
  B.Eng. in Computer Science and Technology — Hangzhou, China
  - Outstanding Graduate of Zhejiang University (2023).

Research Appointments

- **National University of Singapore** — Sep. 2025 – Sep. 2026
  Research Associate, Trustworthy and Secure Software (TSS) Group — Singapore
  - Principal Investigator (PI): Abhik Roychoudhury.
  - Research towards building trustworthy software engineering agents, focusing on reliable code explanations and automated code review.

Selected Research

Towards Building Trustworthy Software Engineering Agents

- **ExplainBench: Evaluating Code Explanations from Agents**
  ASE 2026 | DOI: 10.1145/3832783.3834395
  Shows that issue-resolution efficacy alone does not capture the trustworthiness of an agent's explanations. Introduces behavior-based questions to assess explanation quality and a test-driven audit agent that checks and refines explanations using executable evidence.
- **Code Review Agent Benchmark**
  arXiv:2603.23448; co-first author
  Introduces execution-based code review evaluation by converting human feedback into tests and checking review-guided revisions against the underlying issues. Measures usefulness through verifiable code improvements, beyond comment similarity, and reveals complementary perspectives that motivate human-agent collaboration.

Code LLM Capabilities and Evaluation

- **CatCoder: Repository-Level Code Generation with Relevant Code and Type Context**
  TOSEM | DOI: 10.1145/3779217
  Addresses missing cross-file context in repository-level code generation by combining code retrieval with statically extracted type dependencies. Shows how relevant code and type information complement each other, providing a way to incorporate program structure into LLM generation for statically typed languages.
- **Reasoning Runtime Behavior of a Program with LLM: How Far Are We?**
  ICSE 2025; co-first author | DOI: 10.1109/ICSE55347.2025.00012
  Introduces an execution-grounded framework that evaluates coverage, paths, states and outputs together with consistency across these predictions. Shows why final-answer accuracy alone can mask faulty reasoning about program behavior, providing a finer-grained basis for assessing code LLMs' semantic understanding.
- **Re-Evaluating Code LLM Benchmarks Under Semantic Mutation**
  arXiv:2506.17369
  Develops controlled, semantically similar prompt variants to probe the robustness of code benchmarks. Shows that minor wording changes can alter both model scores and rankings, making prompt sensitivity a necessary consideration when interpreting benchmark results and comparing model capabilities.

Binary Analysis for Software Security

- **PPT4J: Patch Presence Test for Java Binaries**
  ICSE 2024 | DOI: 10.1145/3597503.3639231
  Introduces source-to-bytecode semantic feature matching for detecting known security fixes in distributed Java libraries. Captures fine-grained patch differences that function-level similarity can miss and handles subtle edits and semantically redundant changes, making patch status directly checkable in binary dependencies.